SER321
Assignment1
Virginia Appleton
Repo link : https://github.com/vappleton/ser321-spring26B-vappleto

Part I. Linux Setup.

1. Command line tasks

System used: Linux

```
1: mkdir cli_assignment
2: cd cli_assignment
3: touch "stuff.txt"
4: cat > stuff.txt
5.  wc stuff.txt
6. cat >> stuff.txt
7.  mkdir draft
8. mv stuff.txt draft/
9. touch .secret.txt
10. cp -r draft final
11. mv draft draft.remove
12. mv draft.remove final/
13. ls -lR
14. gzcat NASA_access_log_Aug95.gz | less
15. gunzip NASA_access_log_Aug95.gz
16. mv NASA_access_log_Aug95 logs.txt
17.  mv logs.txt cli_assignment/
18. head -n 100 logs.txt
19. head -n 100 logs.txt > logs_top_100.txt
20. tail -n 100 logs.txt
21. tail -n 100 logs.txt > logs_bottom_100.txt
22. cat logs_top_100.txt logs_bottom_100.txt > logs_snapshot.txt
23. echo "vappleto: This is a great assignment $(date)" >> logs_snapshot.txt
24. less logs.txt
25. cut -d '%' -f 1 < marks.csv | tail -n +2
26. cut -d '%' -f 4 marks.csv | tail -n +2 | sort -n
27. awk -F'%' 'NR>1 {sum += $3; count++} END {print sum/count}' marks.csv
28. awk -F'%' 'NR>1 {sum += $3; count++} END {print sum/count}' marks.csv >
cli_assignment/done.txt
29. mv cli_assignment/done.txt cli_assignment/final/
30.  mv done.txt average.txt
```

2.Some Setup and Examples

2.2 Running examples

Example #1:

1. **Name:** Threads/FirstThread
2. **Example running:**

```
virginiaappleton@iMac FirstThread % gradle run
Starting a Gradle Daemon (subsequent builds will be faster)

> Task :run
Hello from 0 loop=0
Hello from 2 loop=0
Hello from 4 loop=0
Hello from 1 loop=0
Hello from 3 loop=0
Hello from 0 loop=1
Hello from 0 loop=2
Hello from 0 loop=3
Hello from 0 loop=4
Hello from 1 loop=1
Hello from 1 loop=2
Hello from 2 loop=1
Hello from 1 loop=3
Hello from 3 loop=1
Hello from 2 loop=2
Hello from 1 loop=4
Hello from 4 loop=1
Hello from 2 loop=3
Hello from 3 loop=2
Hello from 2 loop=4
Hello from 4 loop=2
Hello from 3 loop=3
Hello from 4 loop=3
Hello from 3 loop=4
Hello from 4 loop=4

[Incubating] Problems report is available at: file:///Users/virginiaappleton/ser321-spring26B-vappleto/ser321e
xamples/Threads/FirstThread/build/reports/problems/problems-report.html

Deprecated Gradle features were used in this build, making it incompatible with Gradle 9.0.

You can use '--warning-mode all' to show the individual deprecation warnings and determine if they come from y
our own scripts or plugins.

For more on this, please refer to https://docs.gradle.org/8.14.3/userguide/command_line_interface.html#sec:com
mand_line_warnings in the Gradle documentation.

BUILD SUCCESSFUL in 29s
2 actionable tasks: 2 executed
virginiaappleton@iMac FirstThread %
```

3. **Purpose:** This program creates multiple threads using the Runnable interface. The number of threads and delay are given as input and each thread runs concurrently printing the messages in a loop.

Example #2:

1. **Name:** Threads/Locks
2. **Example running:**

```
virginiaappleton@iMac Threads % cd Locks
virginiaappleton@iMac Locks % gradle run

> Task :run
Thread0 has lock
Thread0 releasing lock
Thread0 has lock
Thread0 releasing lock
Thread0 has lock
Thread0 releasing lock
Thread0 has lock
Thread0 releasing lock
Thread0 has lock
Thread0 releasing lock
Thread1 has lock
Thread1 releasing lock
Thread1 has lock
Thread1 releasing lock
Thread1 has lock
Thread1 releasing lock
Thread1 has lock
Thread1 releasing lock
Thread1 has lock
Thread1 releasing lock
Thread2 has lock
Thread2 releasing lock
Thread2 has lock
Thread2 releasing lock
Thread2 has lock
Thread2 releasing lock
Thread2 has lock
Thread2 releasing lock
Thread2 has lock
Thread2 releasing lock
Thread3 has lock
Thread3 releasing lock
Thread3 has lock
Thread3 releasing lock
Thread3 has lock
Thread3 releasing lock
Thread3 has lock
Thread3 releasing lock
Thread3 has lock
Thread3 releasing lock
Thread4 has lock
Thread4 releasing lock
Thread4 has lock
Thread4 releasing lock
Thread4 has lock
Thread4 releasing lock
Thread4 has lock
Thread4 releasing lock
Thread4 has lock
Thread4 releasing lock

[Incubating] Problems report is available at: file:///Users/virginiaappleton/ser321-spring26B-vappleto/ser321examples/Threads/Locks/build/reports/problems/problems-report.html

Deprecated Gradle features were used in this build, making it incompatible with Gradle 9.0.

You can use '--warning-mode all' to show the individual deprecation warnings and determine if they come from your own scripts or plugins.

For more on this, please refer to https://docs.gradle.org/8.14.3/userguide/command_line_interface.html#sec:command_line_warnings in the Gradle documentation.

BUILD SUCCESSFUL in 2s
2 actionable tasks: 2 executed
virginiaappleton@iMac Locks %
```

3. **Purpose**
   This example shows thread synchronization using locks to ensure that only one thread executes the critical section at a time, while the other threads wait until the lock is released.

Example #3:

1. **Name:** Threads/Account
2. **Example running:**

```
virginiaappleton@iMac ser321examples % cd Threads/Account
virginiaappleton@iMac Account % gradle run

> Task :run
Transaction started #2
Transaction started #3
Transaction started #1
Transaction started #4
Transaction started #5
Balance is 150

[Incubating] Problems report is available at: file:///Users/virginiaappleton/ser321-spring26B-vappleto/ser321examples/Threads/Account/build/reports/problems/problems-report.html

Deprecated Gradle features were used in this build, making it incompatible with Gradle 9.0.

You can use '--warning-mode all' to show the individual deprecation warnings and determine if they come from your own scripts or plugins.

For more on this, please refer to https://docs.gradle.org/8.14.3/userguide/command_line_interface.html#sec:command_line_warnings in the Gradle documentation.

BUILD SUCCESSFUL in 958ms
2 actionable tasks: 2 executed
virginiaappleton@iMac Account %
```

3. **Purpose:** This example demonstrates thread synchronization using multiple threads that perform deposit operations on shared data (a bank account). The synchronized methods make sure that only one thread updates the account at a time.

2.4 Second system running demo

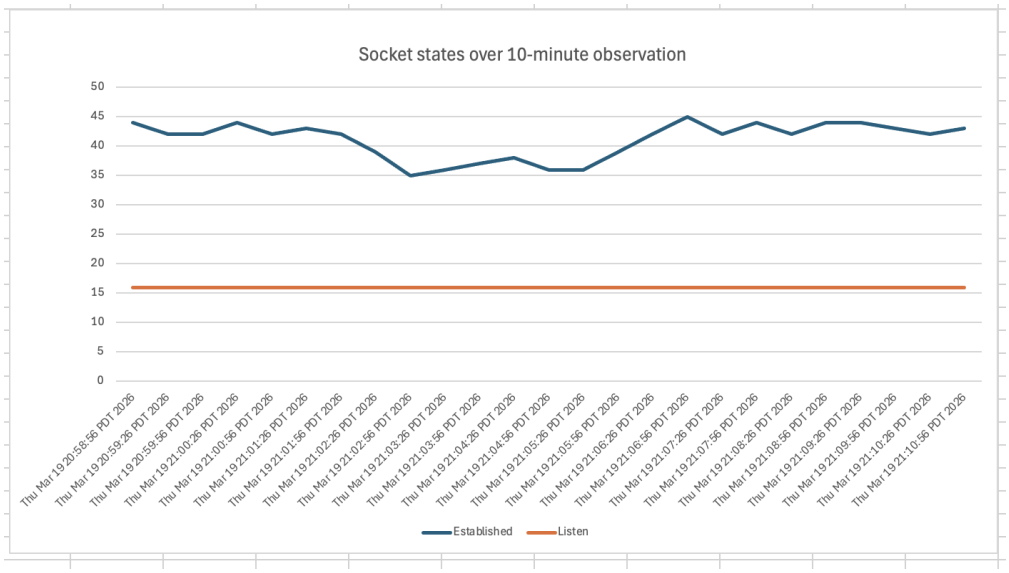
Screencast link: https://youtube.com/shorts/w3Z52-DtRfE?feature=share

Part II Networking

3.1 Understanding TCP network sockets.

Script: while true; do
 echo "$(date),$(netstat -an | grep ESTABLISHED | wc -l),$(netstat -an | grep LISTEN | wc -l)" >> netstat_log.txt
 sleep 30
done

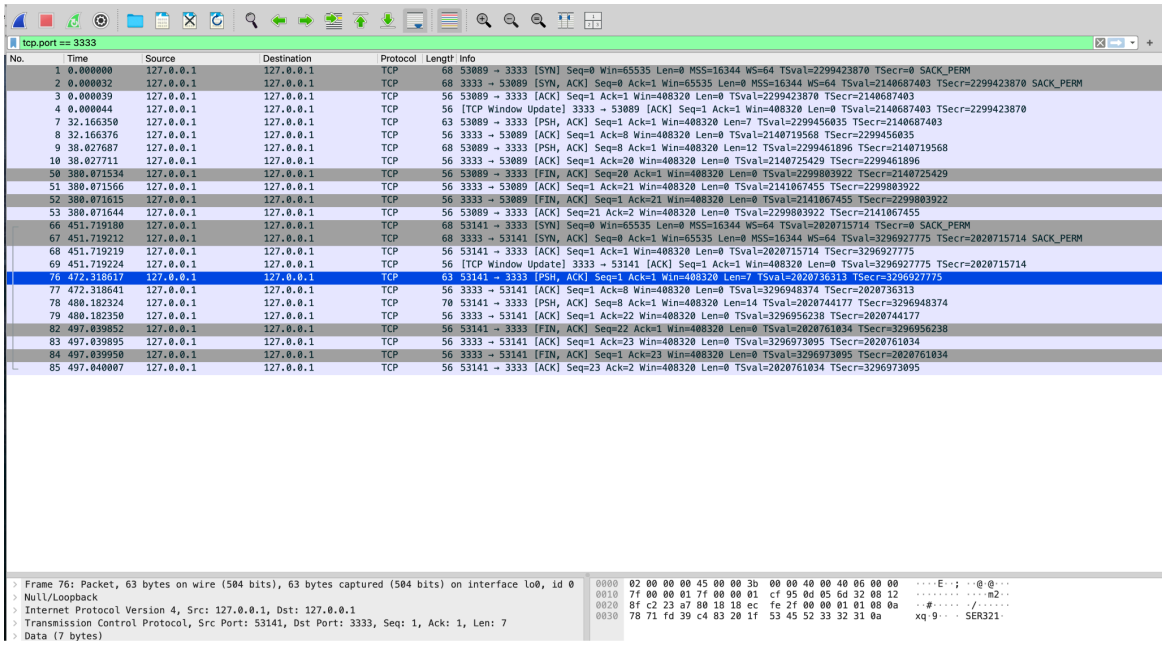
Graph:



The graph shows the number of TCp sockets in the Established and Listen states over a 10-minute observation period, with measurements taken every 30 seconds. The number of established connections varies from 35 to 45 connections, while the number of listening sockets remains constant at 16.

3.2 Sniffing TCP/UDP Traffic

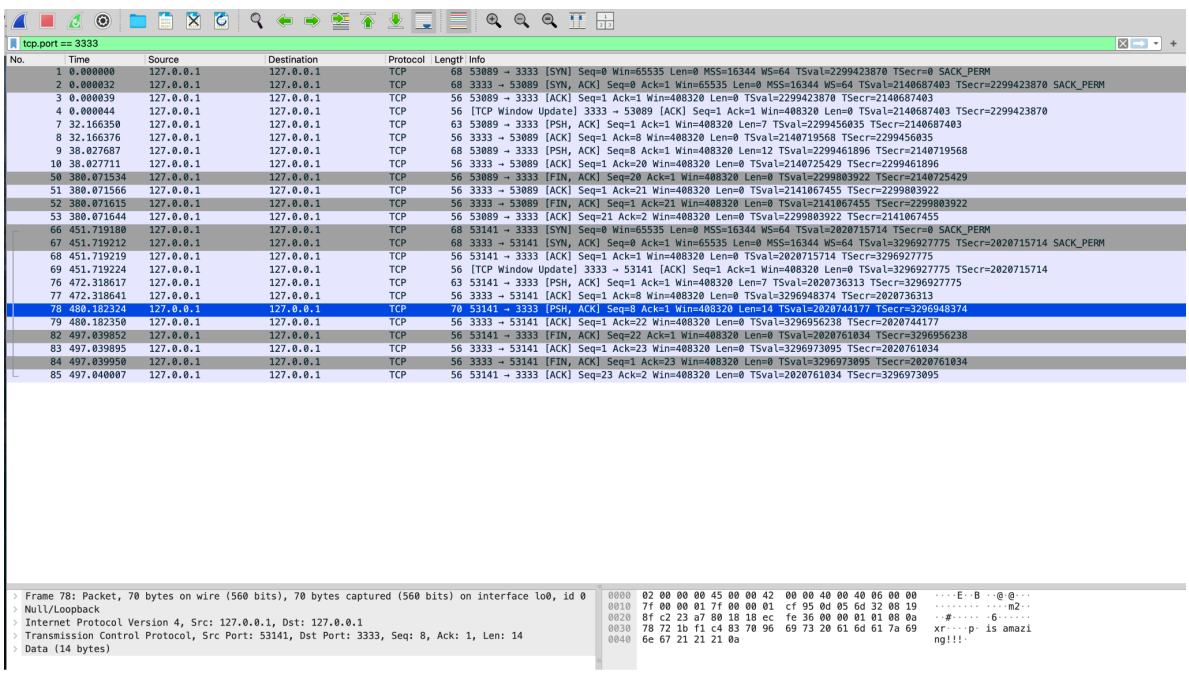
Step 1. TCP



tcp.port == 3333

No.	Time	Source	Destination	Protocol	Length	Info
1	0.000000	127.0.0.1	127.0.0.1	TCP	68	53089 → 3333 [SYN] Seq=0 Win=65535 Len=0 MSS=16344 WS=64 TSval=2299423870 TSecr=0 SACK_PERM
2	0.000032	127.0.0.1	127.0.0.1	TCP	68	3333 → 53089 [SYN, ACK] Seq=0 Ack=1 Win=65535 Len=0 MSS=16344 WS=64 TSval=2140687403 TSecr=2299423870 SACK_PERM
3	0.000039	127.0.0.1	127.0.0.1	TCP	56	53089 → 3333 [ACK] Seq=1 Ack=1 Win=408320 Len=0 TSval=2299423870 TSecr=2140687403
4	0.000044	127.0.0.1	127.0.0.1	TCP	56	[TCP Window Update] 3333 → 53089 [ACK] Seq=1 Ack=1 Win=408320 Len=0 TSval=2140687403 TSecr=2299423870
7	32.166350	127.0.0.1	127.0.0.1	TCP	63	53089 → 3333 [PSH, ACK] Seq=1 Ack=1 Win=408320 Len=7 TSval=2299456035 TSecr=2140687403
8	32.166376	127.0.0.1	127.0.0.1	TCP	56	3333 → 53089 [ACK] Seq=1 Ack=8 Win=408320 Len=0 TSval=2140719568 TSecr=2299456035
9	38.027687	127.0.0.1	127.0.0.1	TCP	68	53089 → 3333 [PSH, ACK] Seq=8 Ack=1 Win=408320 Len=12 TSval=2299461896 TSecr=2140719568
10	38.027711	127.0.0.1	127.0.0.1	TCP	56	3333 → 53089 [ACK] Seq=1 Ack=20 Win=408320 Len=0 TSval=2140725429 TSecr=2299461896
50	380.071534	127.0.0.1	127.0.0.1	TCP	56	53089 → 3333 [FIN, ACK] Seq=20 Ack=1 Win=408320 Len=0 TSval=2299803922 TSecr=2140725429
51	380.071566	127.0.0.1	127.0.0.1	TCP	56	3333 → 53089 [ACK] Seq=1 Ack=21 Win=408320 Len=0 TSval=2141067455 TSecr=2299803922
52	380.071615	127.0.0.1	127.0.0.1	TCP	56	3333 → 53089 [FIN, ACK] Seq=1 Ack=21 Win=408320 Len=0 TSval=2141067455 TSecr=2299803922
53	380.071644	127.0.0.1	127.0.0.1	TCP	56	53089 → 3333 [ACK] Seq=21 Ack=2 Win=408320 Len=0 TSval=2299803922 TSecr=2141067455
66	451.719180	127.0.0.1	127.0.0.1	TCP	68	53141 → 3333 [SYN] Seq=0 Win=65535 Len=0 MSS=16344 WS=64 TSval=2020715714 TSecr=0 SACK_PERM
67	451.719212	127.0.0.1	127.0.0.1	TCP	68	3333 → 53141 [SYN, ACK] Seq=0 Ack=1 Win=65535 Len=0 MSS=16344 WS=64 TSval=3296927775 TSecr=2020715714 SACK_PERM
68	451.719219	127.0.0.1	127.0.0.1	TCP	56	53141 → 3333 [ACK] Seq=1 Ack=1 Win=408320 Len=0 TSval=2020715714 TSecr=3296927775
69	451.719224	127.0.0.1	127.0.0.1	TCP	56	[TCP Window Update] 3333 → 53141 [ACK] Seq=1 Ack=1 Win=408320 Len=0 TSval=3296927775 TSecr=2020715714
76	472.318617	127.0.0.1	127.0.0.1	TCP	63	53141 → 3333 [PSH, ACK] Seq=1 Ack=1 Win=408320 Len=7 TSval=2020736313 TSecr=3296927775
77	472.318641	127.0.0.1	127.0.0.1	TCP	56	3333 → 53141 [ACK] Seq=1 Ack=8 Win=408320 Len=0 TSval=3296948374 TSecr=2020736313
78	480.182324	127.0.0.1	127.0.0.1	TCP	70	53141 → 3333 [PSH, ACK] Seq=8 Ack=1 Win=408320 Len=14 TSval=2020744177 TSecr=3296948374
79	480.182350	127.0.0.1	127.0.0.1	TCP	56	3333 → 53141 [ACK] Seq=1 Ack=22 Win=408320 Len=0 TSval=3296956238 TSecr=2020744177
82	497.039852	127.0.0.1	127.0.0.1	TCP	56	53141 → 3333 [FIN, ACK] Seq=22 Ack=1 Win=408320 Len=0 TSval=2020761034 TSecr=3296956238
83	497.039895	127.0.0.1	127.0.0.1	TCP	56	3333 → 53141 [ACK] Seq=1 Ack=23 Win=408320 Len=0 TSval=3296973095 TSecr=2020761034
84	497.039950	127.0.0.1	127.0.0.1	TCP	56	3333 → 53141 [FIN, ACK] Seq=1 Ack=23 Win=408320 Len=0 TSval=3296973095 TSecr=2020761034
85	497.040007	127.0.0.1	127.0.0.1	TCP	56	53141 → 3333 [ACK] Seq=23 Ack=2 Win=408320 Len=0 TSval=2020761034 TSecr=3296973095

```
> Frame 76: Packet, 63 bytes on wire (504 bits), 63 bytes captured (504 bits) on interface lo0, id 0
> Null/Loopback
> Internet Protocol Version 4, Src: 127.0.0.1, Dst: 127.0.0.1
> Transmission Control Protocol, Src Port: 53141, Dst Port: 3333, Seq: 1, Ack: 1, Len: 7
> Data (7 bytes)

0000  02 00 00 00 45 00 00 3b  00 00 40 00 40 06 00 00   ····E··; ··@·@···
0010  7f 00 00 01 7f 00 00 01  cf 95 0d 05 6d 32 08 12   ........ ····m2··
0020  8f c2 23 a7 80 18 18 ec  fe 2f 00 00 01 01 08 0a   ··#····· ·/ ·····
0030  78 71 fd 39 c4 83 20 1f  53 45 52 33 32 31 0a      xq·9·· · SER321·
```



tcp.port == 3333

No.	Time	Source	Destination	Protocol	Length	Info
1	0.000000	127.0.0.1	127.0.0.1	TCP	68	53089 → 3333 [SYN] Seq=0 Win=65535 Len=0 MSS=16344 WS=64 TSval=2299423870 TSecr=0 SACK_PERM
2	0.000032	127.0.0.1	127.0.0.1	TCP	68	3333 → 53089 [SYN, ACK] Seq=0 Ack=1 Win=65535 Len=0 MSS=16344 WS=64 TSval=2140687403 TSecr=2299423870 SACK_PERM
3	0.000039	127.0.0.1	127.0.0.1	TCP	56	53089 → 3333 [ACK] Seq=1 Ack=1 Win=408320 Len=0 TSval=2299423870 TSecr=2140687403
4	0.000044	127.0.0.1	127.0.0.1	TCP	56	[TCP Window Update] 3333 → 53089 [ACK] Seq=1 Ack=1 Win=408320 Len=0 TSval=2140687403 TSecr=2299423870
7	32.166350	127.0.0.1	127.0.0.1	TCP	63	53089 → 3333 [PSH, ACK] Seq=1 Ack=1 Win=408320 Len=7 TSval=2299456035 TSecr=2140687403
8	32.166376	127.0.0.1	127.0.0.1	TCP	56	3333 → 53089 [ACK] Seq=1 Ack=8 Win=408320 Len=0 TSval=2140719568 TSecr=2299456035
9	38.027687	127.0.0.1	127.0.0.1	TCP	68	53089 → 3333 [PSH, ACK] Seq=8 Ack=1 Win=408320 Len=12 TSval=2299461896 TSecr=2140719568
10	38.027711	127.0.0.1	127.0.0.1	TCP	56	3333 → 53089 [ACK] Seq=1 Ack=20 Win=408320 Len=0 TSval=2140725429 TSecr=2299461896
50	380.071534	127.0.0.1	127.0.0.1	TCP	56	53089 → 3333 [FIN, ACK] Seq=20 Ack=1 Win=408320 Len=0 TSval=2299803922 TSecr=2140725429
51	380.071566	127.0.0.1	127.0.0.1	TCP	56	3333 → 53089 [ACK] Seq=1 Ack=21 Win=408320 Len=0 TSval=2141067455 TSecr=2299803922
52	380.071615	127.0.0.1	127.0.0.1	TCP	56	3333 → 53089 [FIN, ACK] Seq=1 Ack=21 Win=408320 Len=0 TSval=2141067455 TSecr=2299803922
53	380.071644	127.0.0.1	127.0.0.1	TCP	56	53089 → 3333 [ACK] Seq=21 Ack=2 Win=408320 Len=0 TSval=2299803922 TSecr=2141067455
66	451.719180	127.0.0.1	127.0.0.1	TCP	68	53141 → 3333 [SYN] Seq=0 Win=65535 Len=0 MSS=16344 WS=64 TSval=2020715714 TSecr=0 SACK_PERM
67	451.719212	127.0.0.1	127.0.0.1	TCP	68	3333 → 53141 [SYN, ACK] Seq=0 Ack=1 Win=65535 Len=0 MSS=16344 WS=64 TSval=3296927775 TSecr=2020715714 SACK_PERM
68	451.719219	127.0.0.1	127.0.0.1	TCP	56	53141 → 3333 [ACK] Seq=1 Ack=1 Win=408320 Len=0 TSval=2020715714 TSecr=3296927775
69	451.719224	127.0.0.1	127.0.0.1	TCP	56	[TCP Window Update] 3333 → 53141 [ACK] Seq=1 Ack=1 Win=408320 Len=0 TSval=3296927775 TSecr=2020715714
76	472.318617	127.0.0.1	127.0.0.1	TCP	63	53141 → 3333 [PSH, ACK] Seq=1 Ack=1 Win=408320 Len=7 TSval=2020736313 TSecr=3296927775
77	472.318641	127.0.0.1	127.0.0.1	TCP	56	3333 → 53141 [ACK] Seq=1 Ack=8 Win=408320 Len=0 TSval=3296948374 TSecr=2020736313
78	480.182324	127.0.0.1	127.0.0.1	TCP	70	53141 → 3333 [PSH, ACK] Seq=8 Ack=1 Win=408320 Len=14 TSval=2020744177 TSecr=3296948374
79	480.182350	127.0.0.1	127.0.0.1	TCP	56	3333 → 53141 [ACK] Seq=1 Ack=22 Win=408320 Len=0 TSval=3296956238 TSecr=2020744177
82	497.039852	127.0.0.1	127.0.0.1	TCP	56	53141 → 3333 [FIN, ACK] Seq=22 Ack=1 Win=408320 Len=0 TSval=2020761034 TSecr=3296956238
83	497.039895	127.0.0.1	127.0.0.1	TCP	56	3333 → 53141 [ACK] Seq=1 Ack=23 Win=408320 Len=0 TSval=3296973095 TSecr=2020761034
84	497.039950	127.0.0.1	127.0.0.1	TCP	56	3333 → 53141 [FIN, ACK] Seq=1 Ack=23 Win=408320 Len=0 TSval=3296973095 TSecr=2020761034
85	497.040007	127.0.0.1	127.0.0.1	TCP	56	53141 → 3333 [ACK] Seq=23 Ack=2 Win=408320 Len=0 TSval=2020761034 TSecr=3296973095

```
> Frame 78: Packet, 70 bytes on wire (560 bits), 70 bytes captured (560 bits) on interface lo0, id 0
> Null/Loopback
> Internet Protocol Version 4, Src: 127.0.0.1, Dst: 127.0.0.1
> Transmission Control Protocol, Src Port: 53141, Dst Port: 3333, Seq: 8, Ack: 1, Len: 14
> Data (14 bytes)

0000  02 00 00 00 45 00 00 42  00 00 40 00 40 06 00 00   ····E··B ··@·@···
0010  7f 00 00 01 7f 00 00 01  cf 95 0d 05 6d 32 08 19   ........ ····m2··
0020  8f c2 23 a7 80 18 18 ec  fe 36 00 00 01 01 08 0a   ··#····· ·6ˇ·····
0030  78 72 1b f1 c4 83 70 96  69 73 20 61 6d 61 7a 69   xr····p· is amazi
0040  6e 67 21 21 21 0a                                  ng!!!·
```

TCP questions

a) Explain both the commands you used in detail. What did they actually do?

- nc -k -l 3333
  This command starts a netcat server that listens on the TCP port 3333 for incoming connections. The -l option puts netcat into listening mode while the -k option keeps the server running so it accepts multiple connections instead of closing after one.
- nc 127.0.0.1 3333
  This command acts as the client that connects to the server running in the local machine (127.0.0.1) on port 3333. After the connection is established, any text typed into the client terminal is sent to the server over the TCP connection.

b) How many packets were send back and forth so the client/server could send/receive these two lines of data?

2 packets. Each line of input including the newline character generated by pressing Enter, was transmitted in a single TCP packets marked with the PSH, ACK flag

c) How many packets were needed back and forth to capture the whole "process" (starting the communication, ending the communication, sending the lines)?

11 packets. Although the screenshots show more packets, only the packets from the second TCP session (packet No. 66) were considered, since the first session had incomplete input and was restarted. Starting from that point, the full process includes the TCP handshake, data transmission, and connection termination, which together add up to 11 packets.

d) How many bytes is the data (only the data) that was sent from client to server?

21 bytes

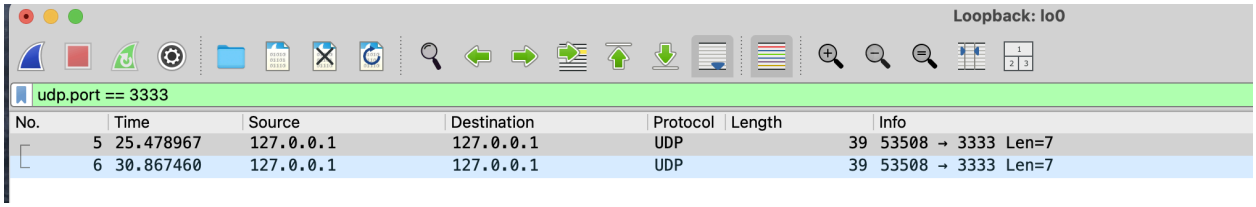
e) How many total bytes went over the wire (back and forth) for the whole process?

Approximately 717 bytes were transmitted over the network for the entire process, including connection setup , data transfer, acknowledgements and connection termination.

f) How much overhead was there. Basically how many bytes was the whole process compared to the actually data that we did send?

The overhead was significantly higher than the actual transmitted data. While only 21 bytes of data were sent, approximately 717 bytes were transmitted in total, which means that around 696 bytes were overhead due to TCP/IP headers, acknowledgments and connection management.

Step 2. UDP



No.	Time	Source	Destination	Protocol	Length	Info
5	25.478967	127.0.0.1	127.0.0.1	UDP	39	53508 → 3333 Len=7
6	30.867460	127.0.0.1	127.0.0.1	UDP	39	53508 → 3333 Len=7

UDP Questions:

a) Explain both the commands you used in detail. What did they actually do?

- nc -k -l -u 3333
  This command starts a UDP server listening on port 3333. The -u option specifies UDP instead of TCP, the -l option puts netcat into listening mode and the -k option keeps the server running to receive multiple messages.
- nc -u 127.0.0.1 3333
  This command starts a UDP client that sends data to the local machine on port 3333. Once the connection is made, any text typed into the client terminal is sent as UDP datagrams to the server.

b) How many packets were needed to capture those 2 lines?

2 packets

c) How many packets were needed to capture the whole "process" (starting the communication, ending the communication)?

2 packets. Since UDP doesn't establish or terminate connections, only the data packets were sent.

d) How many bytes is the data (only the data) that was sent?

14 bytes

e) How many total bytes went over the wire?

Approximately 78 bytes were transmitted over the wire.

f) Basically how many bytes was the whole process compared to the actually data that we did send?

The whole process was 78 bytes, while only 14 bytes were actual data. This means that about 64 bytes were overhead due to protocol headers.

g) What is the difference in relative overhead between UDP and TCP and why? Specifically, what kind of information was exchanged in TCP that was not exchanged in UDP? Show the relative parts of the packet traces.

UDP has significantly less overhead than TCP because it does not establish a connection, send acknowledgments or manage the termination of the connection. In TCP, additional packets are required for the three-way-handshale (SYN-ACK,ACK), acknowledgments for each data segment, and connection termination (FIN, ACK). These control packets increase the total overhead.

In contrast, UDP simply sends datagrams without any connection setup, and because of this, only the data packets are transmitted. This reduces both the total number of packets and overall overhead. This difference can be seen in the packet traces, where TCP included multiple control packets while UDP only included the data packets.

Part 3.3 Running client servers in different ways

3.3.1 Running things locally

Command used on server: gradle sockServer
Command used on client: gradle SockClient -Phost=127.0.0.1 -Pmessage=hello -Pnumber=5
gradle SockClient -Phost=127.0.0.1 -Pmessage=Stardust -Pnumber=8
gradle SockClient -Phost=127.0.0.1 -Pmessage=Stardust -Pnumber=6

Client window:
1st connection

```
2 actionable tasks: 1 executed, 1 up-to-date
virginiaappleton@iMac JavaSimpleSock2 % gradle SockClient -Phost=127.0.0.1 -Pmessage=hello -Pnumber=5
Starting a Gradle Daemon, 1 busy and 1 stopped Daemons could not be reused, use --status for details

> Task :SocketClient
Got it!

[Incubating] Problems report is available at: file:///Users/virginiaappleton/Desktop/ser321-spring26B-v
s/problems/problems-report.html

Deprecated Gradle features were used in this build, making it incompatible with Gradle 9.0.

You can use '--warning-mode all' to show the individual deprecation warnings and determine if they come

For more on this, please refer to https://docs.gradle.org/8.14.3/userguide/command_line_interface.html#

BUILD SUCCESSFUL in 10s
2 actionable tasks: 1 executed, 1 up-to-date
```

2nd connection:

```
virginiaappleton@iMac JavaSimpleSock2 % gradle SockClient -Phost=127.0.0.1 -Pmessage=Stardust -Pnumber=8


> Task :SocketClient
Got it!

[Incubating] Problems report is available at: file:///Users/virginiaappleton/Desktop/ser321-spring26B-vapplet
s/problems/problems-report.html

Deprecated Gradle features were used in this build, making it incompatible with Gradle 9.0.

You can use '--warning-mode all' to show the individual deprecation warnings and determine if they come from

For more on this, please refer to https://docs.gradle.org/8.14.3/userguide/command_line_interface.html#sec:co

BUILD SUCCESSFUL in 832ms
2 actionable tasks: 1 executed, 1 up-to-date
```

3rd connection:

```
virginiaappleton@iMac JavaSimpleSock2 % gradle SockClient -Phost=127.0.0.1 -Pmessage=Stardust -Pnumber=6


> Task :SocketClient
Got it!

[Incubating] Problems report is available at: file:///Users/virginiaappleton/Desktop/ser321-spring26B-vapp
s/problems/problems-report.html

Deprecated Gradle features were used in this build, making it incompatible with Gradle 9.0.

You can use '--warning-mode all' to show the individual deprecation warnings and determine if they come fr

For more on this, please refer to https://docs.gradle.org/8.14.3/userguide/command_line_interface.html#sec

BUILD SUCCESSFUL in 829ms
```

Server window:

```
BUILD SUCCESSFUL in 2s
1 actionable task: 1 executed
virginiaappleton@iMac JavaSimpleSock2 % gradle SockServer

> Task :SocketServer
Server ready for 3 connections
Server waiting for a connection
Received the String hello
Received the Integer 5
Server waiting for a connection
Received the String Stardust
Received the Integer 8
Server waiting for a connection
Received the String Stardust
Received the Integer 6

[Incubating] Problems report is available at: file:///Users/virginiaappleton/Desktop/ser321-spring26B-vappleto/ser321examples/
Sockets/JavaSimpleSock2/build/reports/problems/problems-report.html

Deprecated Gradle features were used in this build, making it incompatible with Gradle 9.0.

You can use '--warning-mode all' to show the individual deprecation warnings and determine if they come from your own scripts
or plugins.

For more on this, please refer to https://docs.gradle.org/8.14.3/userguide/command_line_interface.html#sec:command_line_warnin
gs in the Gradle documentation.

BUILD SUCCESSFUL in 8m 36s
2 actionable tasks: 1 executed, 1 up-to-date
virginiaappleton@iMac JavaSimpleSock2 %
```

Screencast link: https://youtu.be/YTKXCz5uXLA

3.3.2. Server on AWS

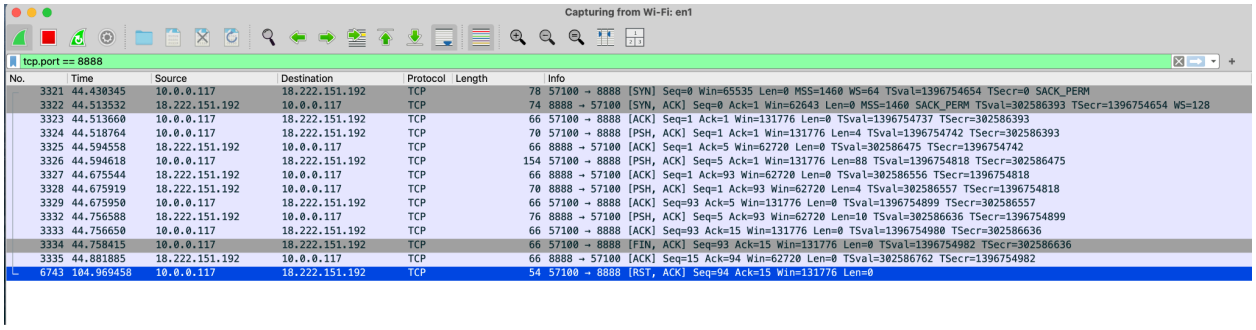
Commands on AWS terminal

```
[ec2-user@ip-172-31-34-94 Sockets]$ cd JavaSimpleSock2
[ec2-user@ip-172-31-34-94 JavaSimpleSock2]$ gradle SockServer

> Task :SocketServer
Server ready for 3 connections
Server waiting for a connection
Received the String hello
Received the Integer 8
Server waiting for a connection
Received the String Stardust
Received the Integer 8
Server waiting for a connection
<=========----> 75% EXECUTING [5m 47s]
> :SocketServer
```

Commands on client (local machine)

```
virginiaappleton@iMac JavaSimpleSock2 % gradle SockClient -Phost=ec2-18-222-151-192.us-east-2.compute.amazonaws.com
-Pport=8888 -Pmessage=hello -Pnumber=8

> Task :SocketClient
Got it!

[Incubating] Problems report is available at: file:///Users/virginiaappleton/Desktop/ser321-spring26B-vappleto/ser32
1examples/Sockets/JavaSimpleSock2/build/reports/problems/problems-report.html

Deprecated Gradle features were used in this build, making it incompatible with Gradle 9.0.

You can use '--warning-mode all' to show the individual deprecation warnings and determine if they come from your ow
n scripts or plugins.

For more on this, please refer to https://docs.gradle.org/8.14.3/userguide/command_line_interface.html#sec:command_l
ine_warnings in the Gradle documentation.

BUILD SUCCESSFUL in 1s
2 actionable tasks: 1 executed, 1 up-to-date
virginiaappleton@iMac JavaSimpleSock2 % gradle SockClient -Phost=ec2-18-222-151-192.us-east-2.compute.amazonaws.com
-Pport=8888 -Pmessage=Stardust -Pnumber=8

> Task :SocketClient
Got it!

[Incubating] Problems report is available at: file:///Users/virginiaappleton/Desktop/ser321-spring26B-vappleto/ser32
1examples/Sockets/JavaSimpleSock2/build/reports/problems/problems-report.html

Deprecated Gradle features were used in this build, making it incompatible with Gradle 9.0.

You can use '--warning-mode all' to show the individual deprecation warnings and determine if they come from your ow
n scripts or plugins.

For more on this, please refer to https://docs.gradle.org/8.14.3/userguide/command_line_interface.html#sec:command_l
ine_warnings in the Gradle documentation.

BUILD SUCCESSFUL in 1s
2 actionable tasks: 1 executed, 1 up-to-date
virginiaappleton@iMac JavaSimpleSock2 %
```

Wireshark:



No.	Time	Source	Destination	Protocol	Length	Info
3321	44.430345	10.0.0.117	18.222.151.192	TCP	78	57100 → 8888 [SYN] Seq=0 Win=65535 Len=0 MSS=1460 WS=64 TSval=1396754654 TSecr=0 SACK_PERM
3322	44.513532	18.222.151.192	10.0.0.117	TCP	74	8888 → 57100 [SYN, ACK] Seq=0 Ack=1 Win=62643 Len=0 MSS=1460 SACK_PERM TSval=302586393 TSecr=1396754654 WS=128
3323	44.513660	10.0.0.117	18.222.151.192	TCP	66	57100 → 8888 [ACK] Seq=1 Ack=1 Win=131776 Len=0 TSval=1396754737 TSecr=302586393
3324	44.518764	10.0.0.117	18.222.151.192	TCP	70	57100 → 8888 [PSH, ACK] Seq=1 Ack=1 Win=131776 Len=4 TSval=1396754742 TSecr=302586393
3325	44.594558	18.222.151.192	10.0.0.117	TCP	66	8888 → 57100 [ACK] Seq=1 Ack=5 Win=62720 Len=0 TSval=302586475 TSecr=1396754742
3326	44.594618	10.0.0.117	18.222.151.192	TCP	154	57100 → 8888 [PSH, ACK] Seq=5 Ack=1 Win=131776 Len=88 TSval=1396754818 TSecr=302586475
3327	44.675544	18.222.151.192	10.0.0.117	TCP	66	8888 → 57100 [ACK] Seq=1 Ack=93 Win=62720 Len=0 TSval=302586556 TSecr=1396754818
3328	44.675919	18.222.151.192	10.0.0.117	TCP	70	8888 → 57100 [PSH, ACK] Seq=1 Ack=93 Win=62720 Len=4 TSval=302586557 TSecr=1396754818
3329	44.675950	10.0.0.117	18.222.151.192	TCP	66	57100 → 8888 [ACK] Seq=93 Ack=5 Win=131776 Len=0 TSval=1396754899 TSecr=302586557
3332	44.756588	18.222.151.192	10.0.0.117	TCP	76	8888 → 57100 [PSH, ACK] Seq=5 Ack=93 Win=62720 Len=10 TSval=302586636 TSecr=1396754899
3333	44.756650	10.0.0.117	18.222.151.192	TCP	66	57100 → 8888 [ACK] Seq=93 Ack=15 Win=131776 Len=0 TSval=1396754980 TSecr=302586636
3334	44.758415	10.0.0.117	18.222.151.192	TCP	66	57100 → 8888 [FIN, ACK] Seq=93 Ack=15 Win=131776 Len=0 TSval=1396754982 TSecr=302586636
3335	44.881885	18.222.151.192	10.0.0.117	TCP	66	8888 → 57100 [ACK] Seq=15 Ack=94 Win=62720 Len=0 TSval=302586762 TSecr=1396754982
6743	104.969458	10.0.0.117	18.222.151.192	TCP	54	57100 → 8888 [RST, ACK] Seq=94 Ack=15 Win=131776 Len=0

What changed?

When I ran everything locally, both client and the server used 127.0.0.1, so all the traffic stayed within my own computer. In Wireshark, I captured this using the loopback interface (lo0).

When I moved the server to AWS, the communication was no longer local. The client connected to the EC2 public IP instead, and I could see traffic going between my local IP and the AWS IP. Because of this, I had to switch Wireshark to capture on my wi-fi interface (en1) instead of the loopback interface. The filter tcp.port ==8888 stayed the same.

For the Gradle commands, nothing changed on the server side, but I ran it on the EC2 instance instead of locally. On the client side, the command changed because I replaced my local IP 127.0.0.1 with the AWS public DNS/IP and made sure to include the correct port. I also had to make sure the EC2 security group allowed traffic on that port, otherwise the connection would have failed. Overall the main difference is that the local setup uses external communication while the AWS setup sends data over the network, but the TCP behavior stays the same. (handshake, data transfer, etc).

3.3.3. Client on AWS

Consider the case you want to run your server locally (so on your home computer) and your client on AWS (you do not have to do this but you can try). Does this work without issues? Can you do it in the same way as in 3.3.2? Why or why not? What is different?

I think this would be more complicated because when the server is on AWS, it has a public IP address that can be accessed from anywhere so the client can connect easily without issues. However, when the server is running locally on my home computer, it does not have a public IP that is directly accessible from the internet. Most home networks use a router with private IP addresses, so an external machine like an AWS client, cannot reach the server by default. Therefore, the main difference is that AWS provides a publicly accessible server while a local machine runs typically behind a private network.

3.3.4. Client on AWS 2

In this context also explain how the differences in local IP addresses, how your router plays into all of this. Why can you easily reach your server on AWS with a client running in your local network but not as easily in the other direction (using a client on AWS or a computer on an external network and connecting to a server that is within the local network)? And what can you do to reach your server in your local network if you want to reach it from outside your network (you do not have to do that)? What is the "issue" if you want to run your server locally and reach it from the "outside world"?

My computer on a home network uses a private IP address which is not visible to the outside world. My router sits between my local network and the Internet and uses a public IP to communicate externally. Because of this, it is easy for a local client to connect to a server on AWS, since AWS provides a public IP address that's accessible from anywhere. However, going in the other direction is harder since a client on AWS cannot directly connect to my local server because my router blocks incoming connections by default.

What could make this work is setting up port forwarding on my router so that incoming traffic on a specific port is forwarded to my local machine. I might also need to adjust firewall settings to allow that traffic.
In conclusion, the main issue with running a server locally and a client on AWS is that local machines are behind a private network and are not directly accessible from the outside world, unlike AWS servers which are designed to be publicly reachable.